Artefact Report

Swoyam Pokharel
Shiv Kumar Yadav

Date created: 2026-03-03
Date modified: 2026-03-03

Contents

0.1. Artefact Report ... 1
1. 1. Introduction ... 1
1.1. 1.1 Scope and Evidence Base ... 2
2. 2. OopsMate Engine ... 3
2.1. 2.1 Engine Structure ... 3
2.2. 2.2 Core Domain Types and Position State ... 4
2.3. 2.3 Search and Caching ... 6
2.4. 2.4 Runtime Control Flow ... 8
3. 3. nnuebie NNUE Inference Crate ... 10
3.1. 3.1 Integration and Dataflow ... 10
3.2. 3.2 Core NNUE Structures ... 12
3.3. 3.3 Incremental Update Sequence ... 15
4. 4. Strikes Attack-Generation Crate ... 16
4.1. 4.1 Attack-Generation Design ... 16
4.2. 4.2 Rays, Paths, and Lookup Pipeline ... 17
4.3. 4.3 Initialization Strategy ... 19
5. 5. Utilities Support Crate ... 20
5.1. 5.1 Utility Class Structure ... 20
5.2. 5.2 Parsing and Rendering Activities ... 21
6. 6. Cross-Cutting Architecture ... 23
6.1. 6.1 System Architecture View ... 23
6.2. 6.2 Dependency View ... 23
7. 7. Conclusion ... 25

0.1. Artefact Report

1. 1. Introduction

This artefact report documents the software design and implementation structure of the FYP chess-engine workbench. The project is organized around four technical units:

- `OopsMate`: the Rust chess engine executable and search framework.

- `Strikes` : precomputed attack and ray tables for fast move generation.
- `Utilities` : common notation and board-display helpers.
- `nnuebie` : a standalone Rust NNUE inference crate for chess evaluation.

The report presents architecture, key data/control flows, and implementation-level diagrams generated from the current codebase and maintained in `artefact-report/artefact-diagrams` .

1.1. 1.1 Scope and Evidence Base

The technical statements in this report are aligned with the current repository layout and module boundaries, especially:

- `src/OopsMate/Cargo.toml` and `src/OopsMate/src/` for engine composition.
- `src/OopsMate/crates/strikes/src/` and `src/OopsMate/crates/utilities/src/` for support crates.
- `src/nnuebie/README.md` and `src/nnuebie/src/` for NNUE design and API intent.

The design objective is clean separation of responsibilities: search and protocol handling in the engine, move legality infrastructure in `strikes` , utility abstractions in `utilities` , and high-performance neural evaluation in `nnuebie` .

2. 2. OopsMate Engine

`OopsMate` is the primary engine crate (`oops_mate`) that provides UCI I/O, position state management, move generation, search, and transposition-table support.

2.1. 2.1 Engine Structure

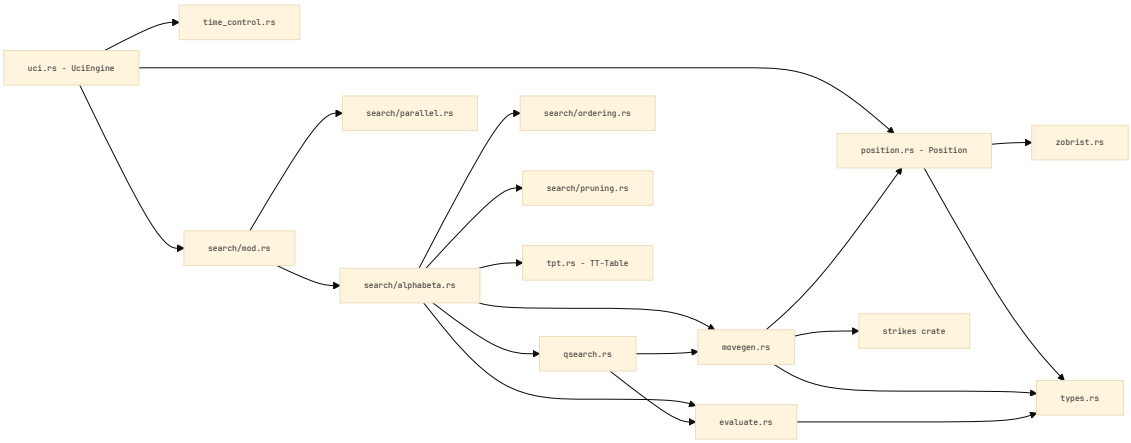

Figure 1. Component-level architecture of the OopsMate engine

The component diagram reflects the modular split in `src/OopsMate/src/`, where protocol, search, movegen, and board-state logic are separate modules that interact through explicit data types.

2.2. 2.2 Core Domain Types and Position State

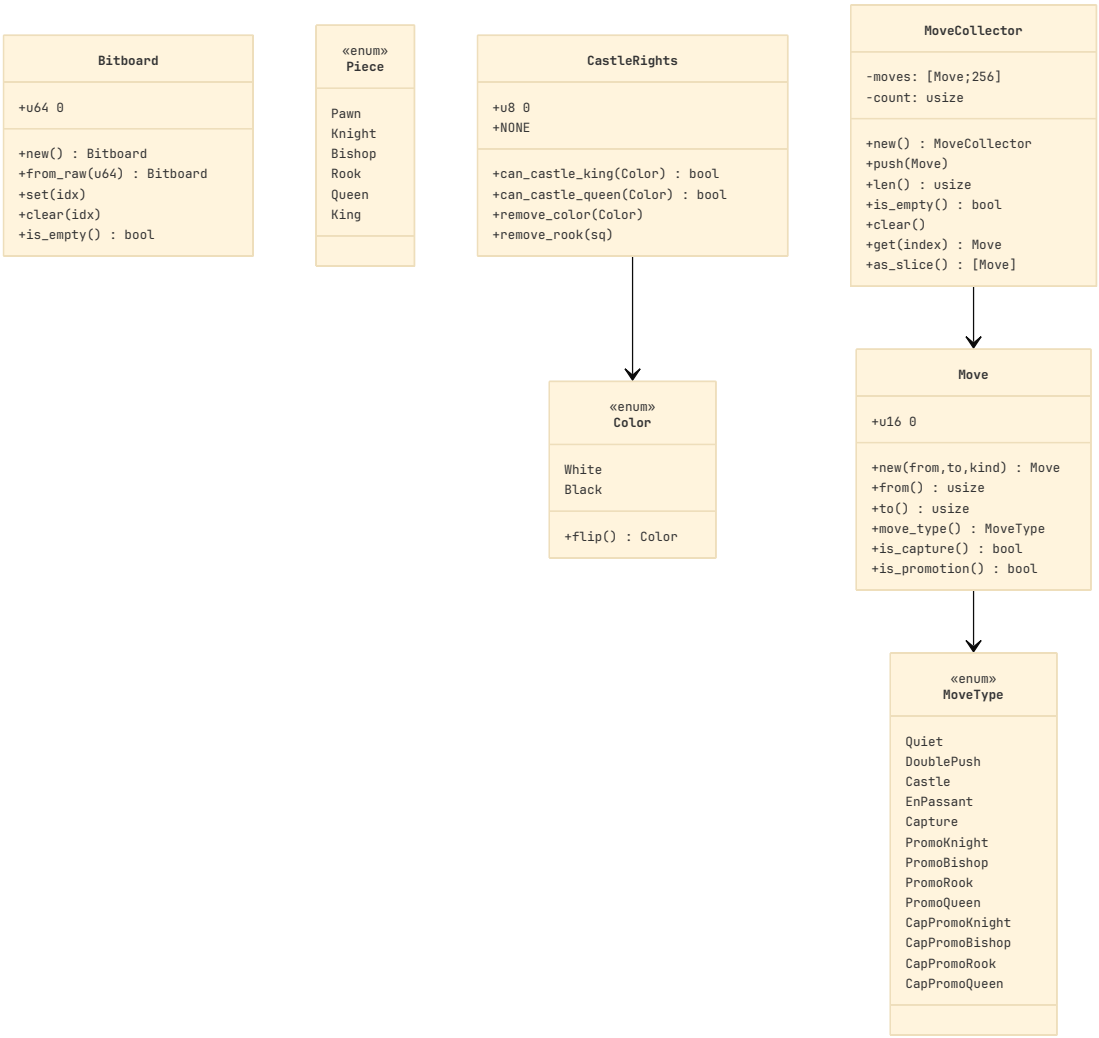

Figure 2. Core move, piece, and color domain types

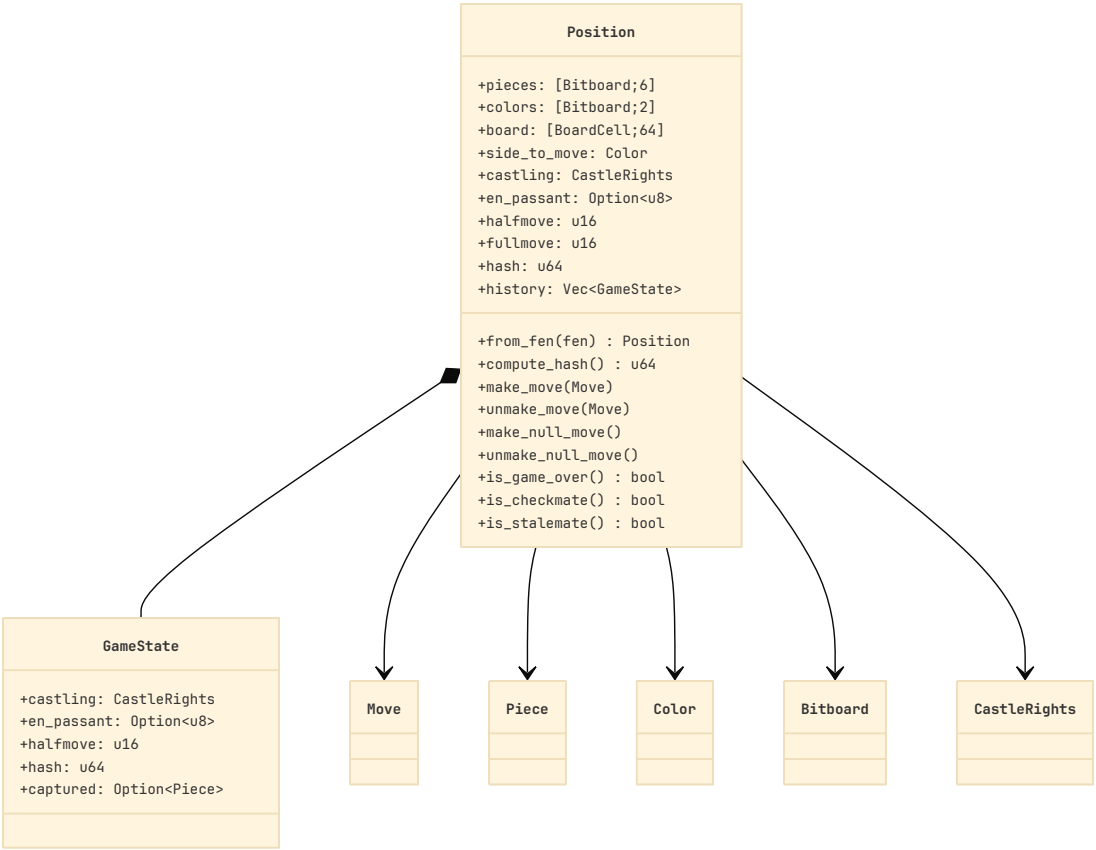

Figure 3. Position representation and board-state responsibilities

The position model centralizes game state (pieces, side to move, castling, en passant, and counters), enabling deterministic evaluation and reproducible search traversal.

2.3. 2.3 Search and Caching

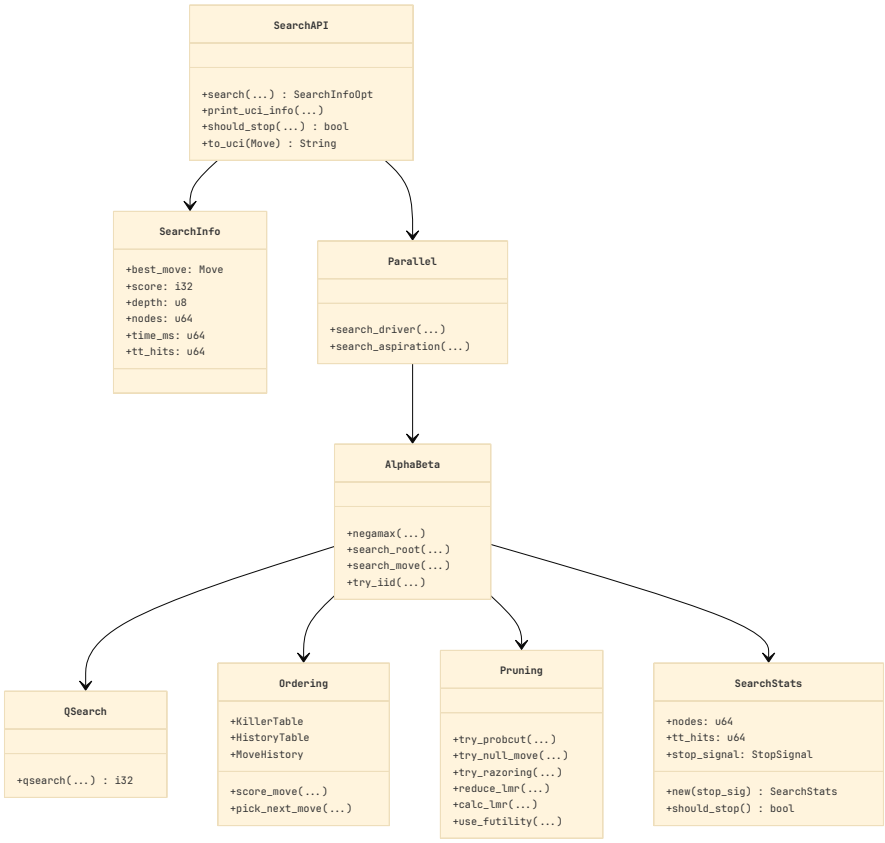

Figure 4. Search subsystem classes and interactions

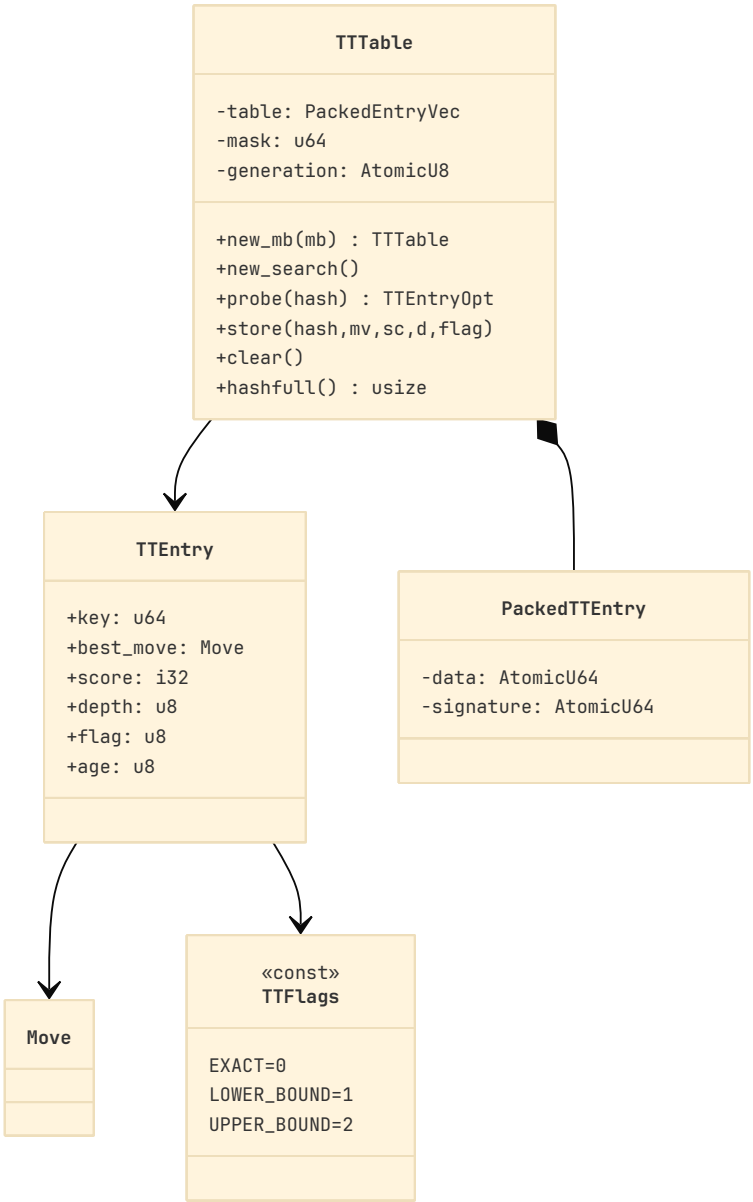

Figure 5. TT-table structures and lookup lifecycle

The search pipeline combines iterative deepening, alpha-beta style pruning infrastructure, and shared transposition-table access. This supports stronger move ordering and reduced repeated work across similar positions.

2.4. 2.4 Runtime Control Flow

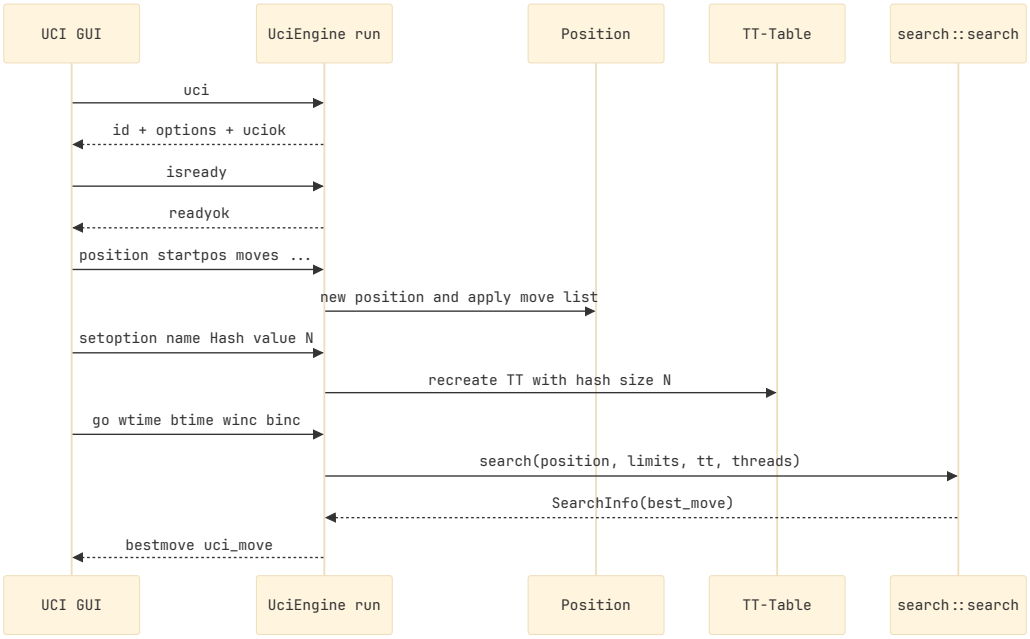

Figure 6. UCI command handling and engine loop interaction

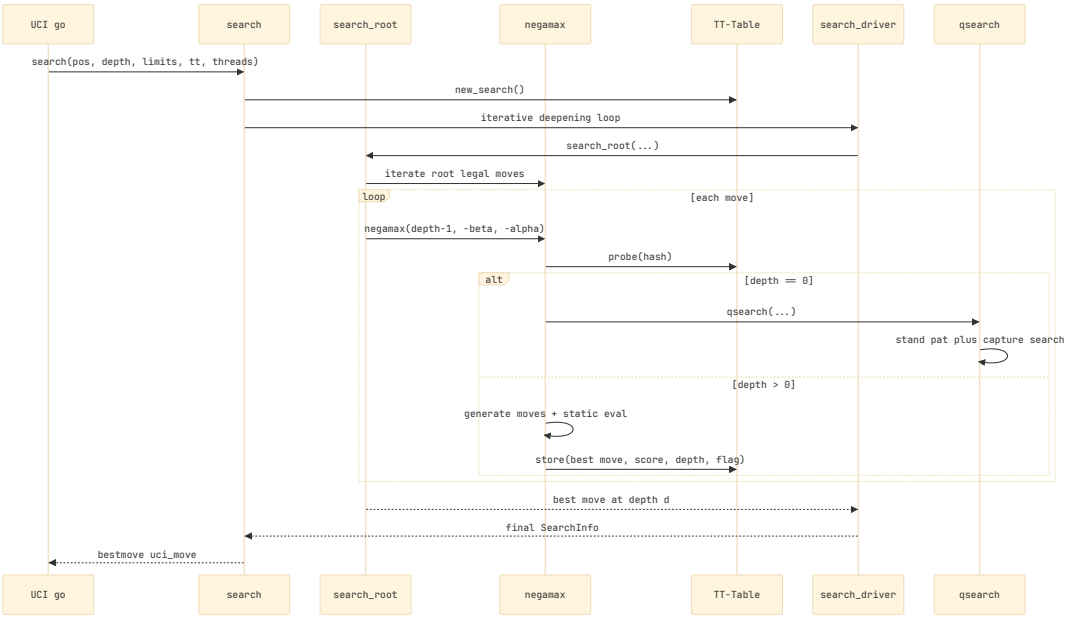

Figure 7. Search invocation flow from input command to best-move output

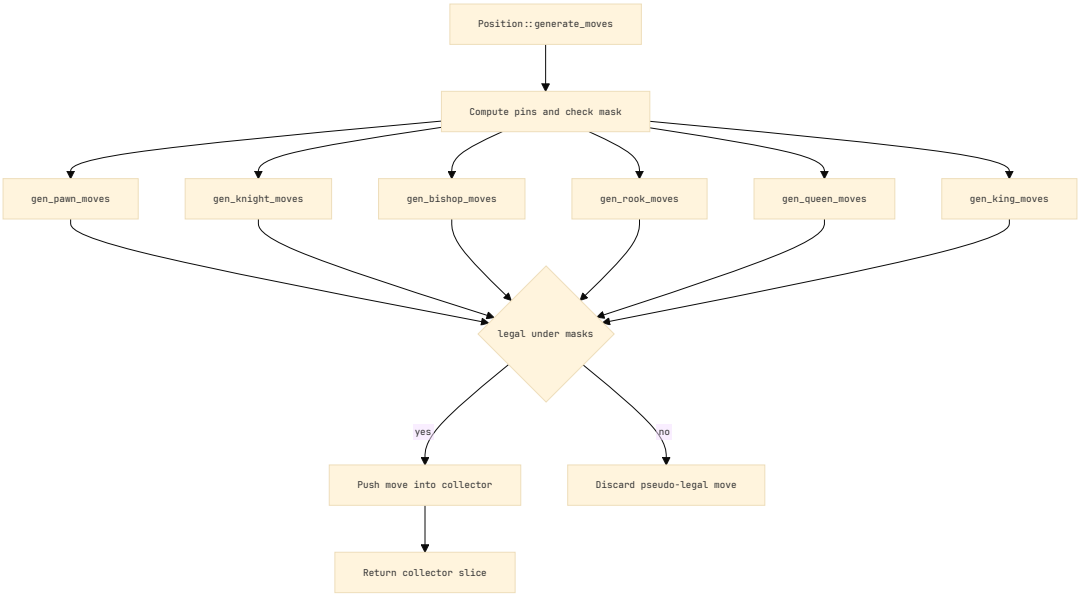

Figure 8. Move-generation activity from position to legal move list

At runtime, `main.rs` initializes attack-table warmup and search parameters, then enters the UCI loop. The resulting flow is suitable for tournament-style integration with GUI frontends and testing harnesses.

3. 3. nnuebie NNUE Inference Crate

`nnuebie` is an independent Rust crate focused on high-throughput NNUE inference for chess engines. It follows the efficiently updatable design: sparse feature extraction, accumulator caching, incremental updates, and quantized forward propagation.

3.1. 3.1 Integration and Dataflow

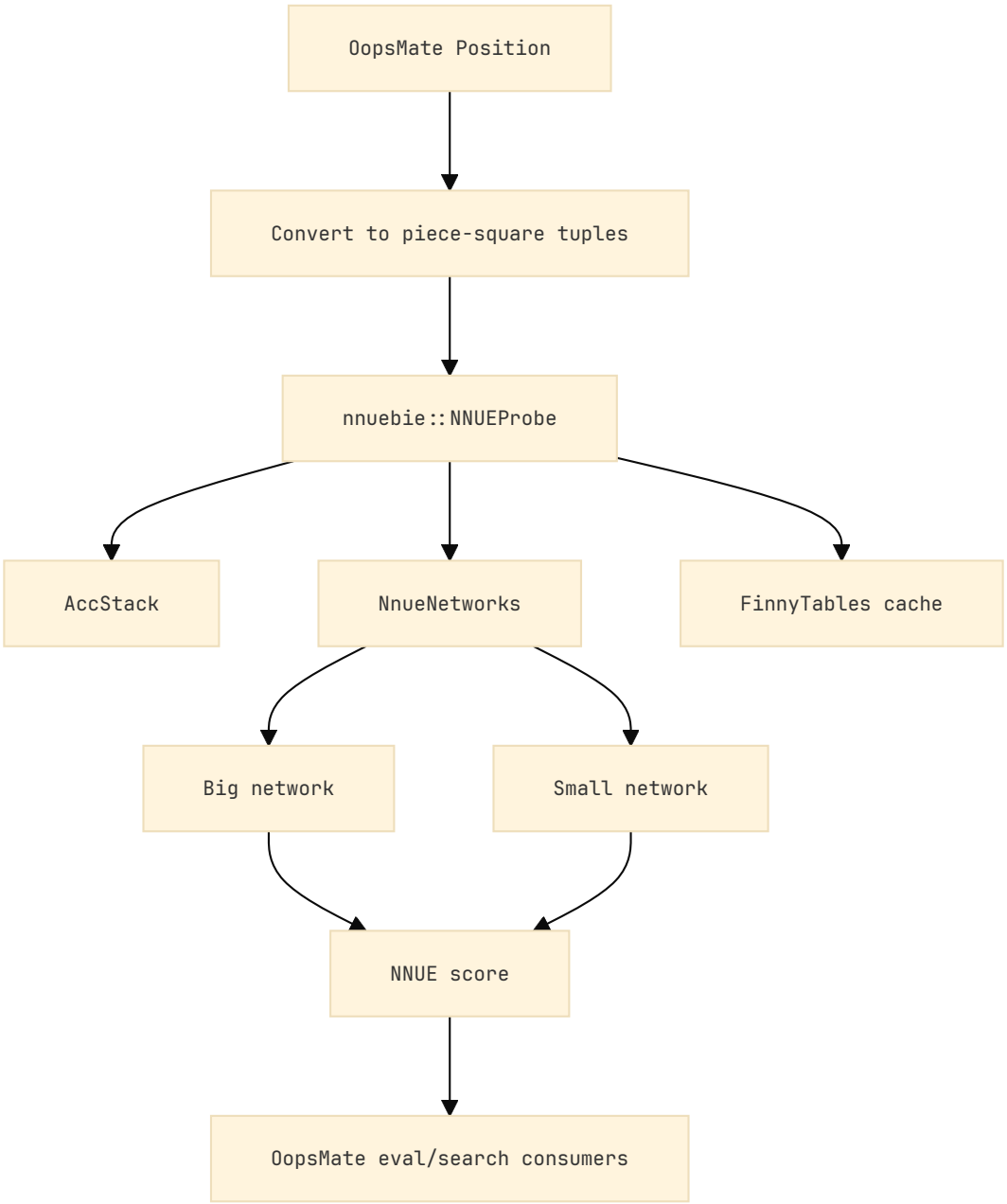

Figure 9. Integration boundary between engine state and nnuebie evaluation

 Artefact Report

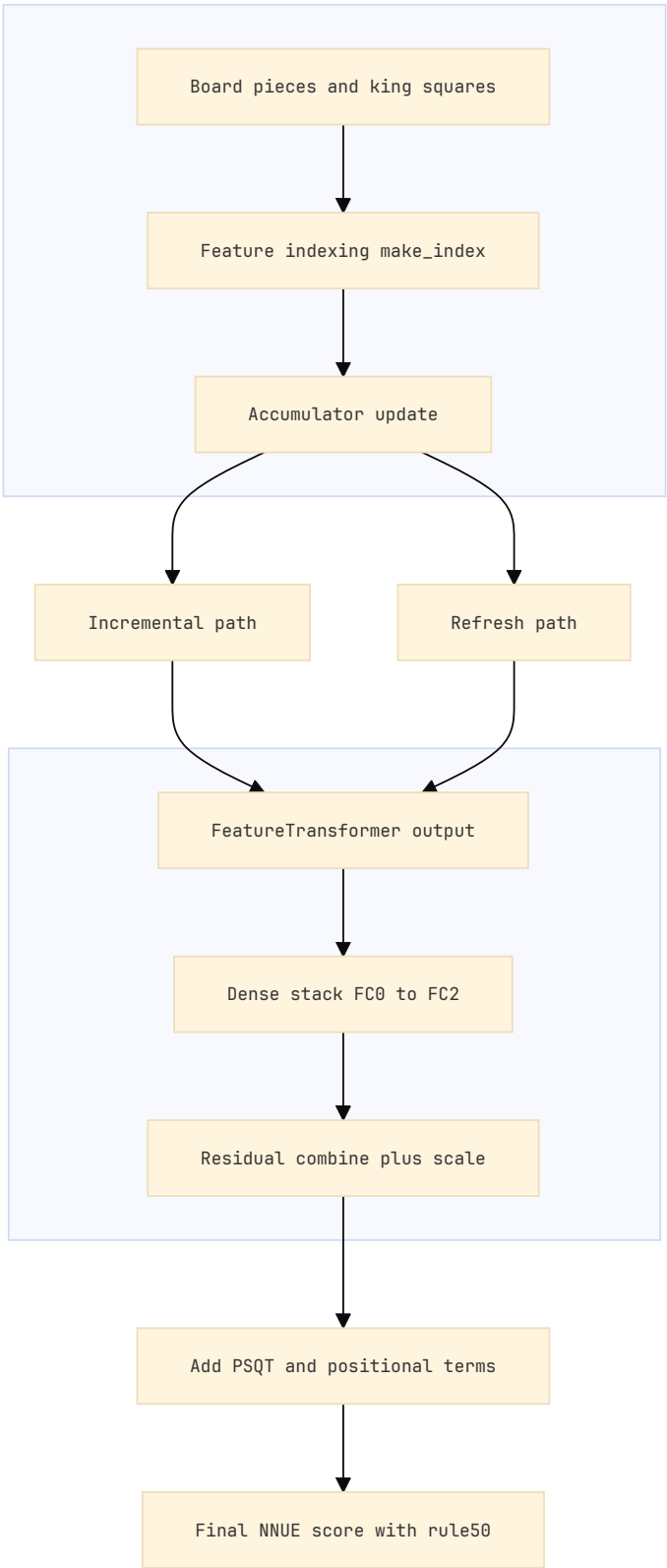

Figure 10. NNUE dataflow from board features to scalar evaluation score

The integration design allows an engine to provide piece-square state while the crate handles network loading, incremental accumulator maintenance, and final centipawn-like output.

3.2. 3.2 Core NNUE Structures

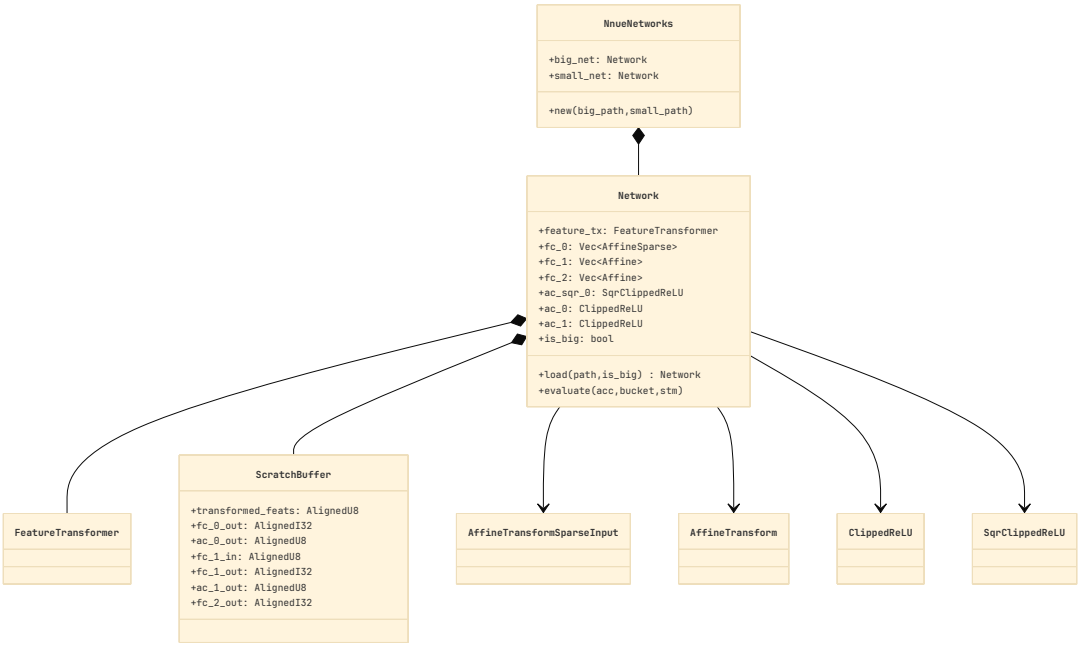

Figure 11. Network-level structures for loaded NNUE parameters

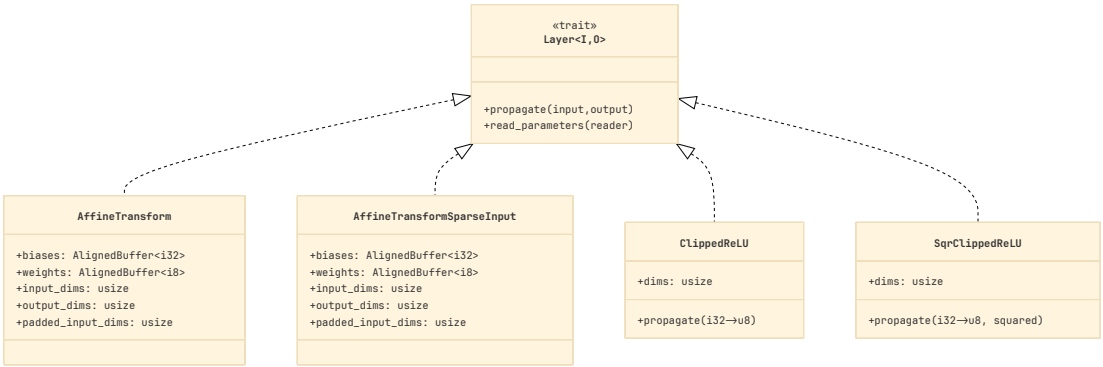

Figure 12. Layer composition used in integer forward propagation

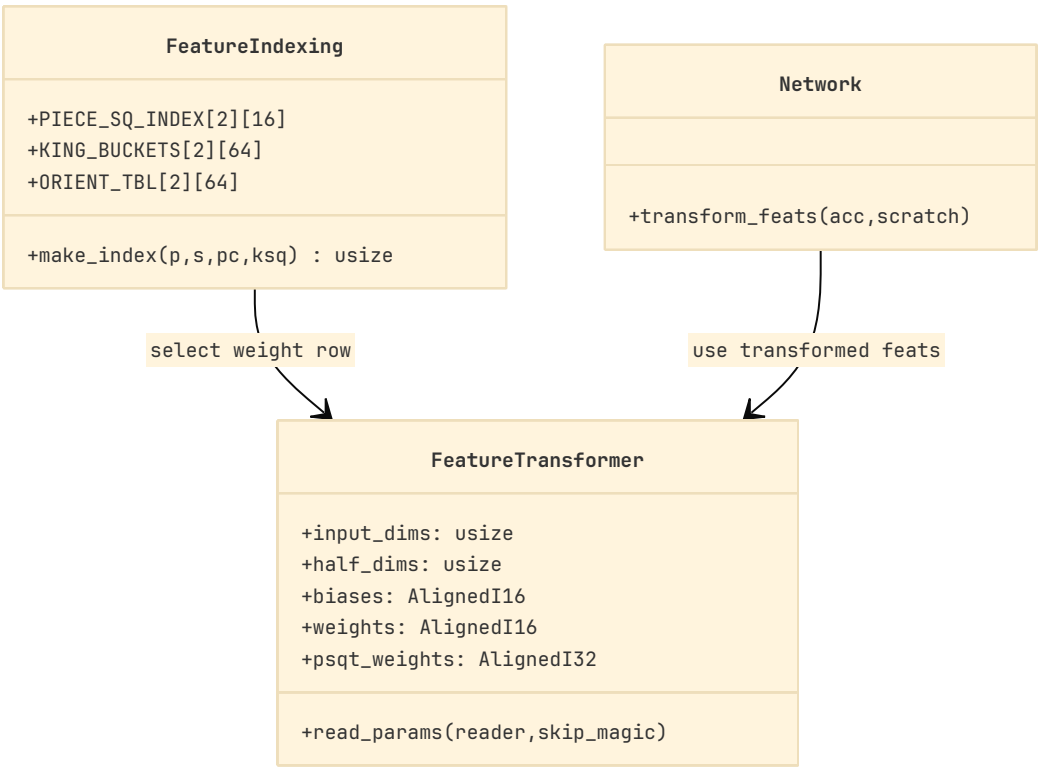

Figure 13. Feature-transformer responsibilities and update path

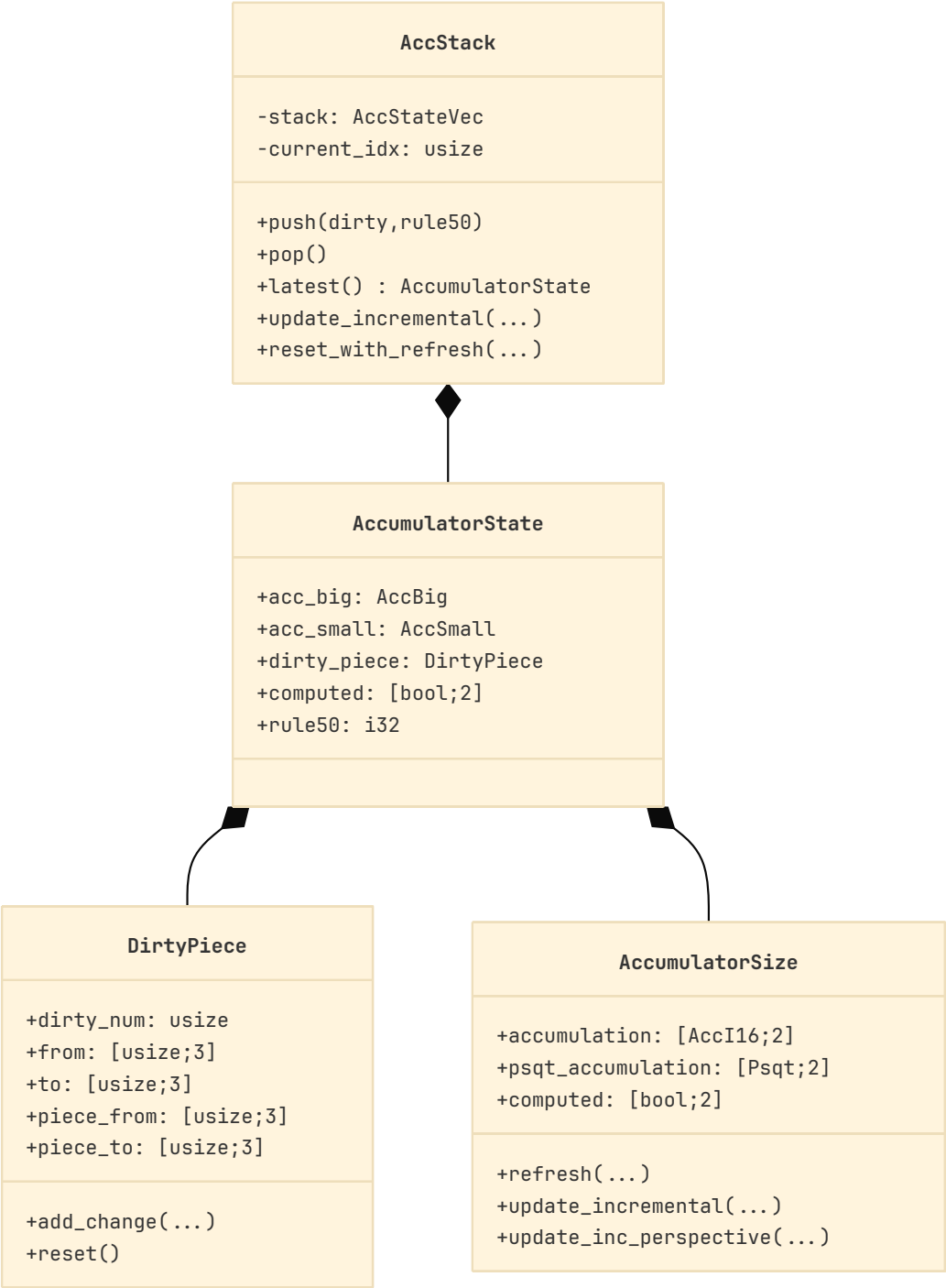

Figure 14. Accumulator state and incremental update semantics

These structures correspond to the HalfKA-style NNUE processing model described in the crate documentation (`src/nnuebie/README.md`).

3.3. 3.3 Incremental Update Sequence

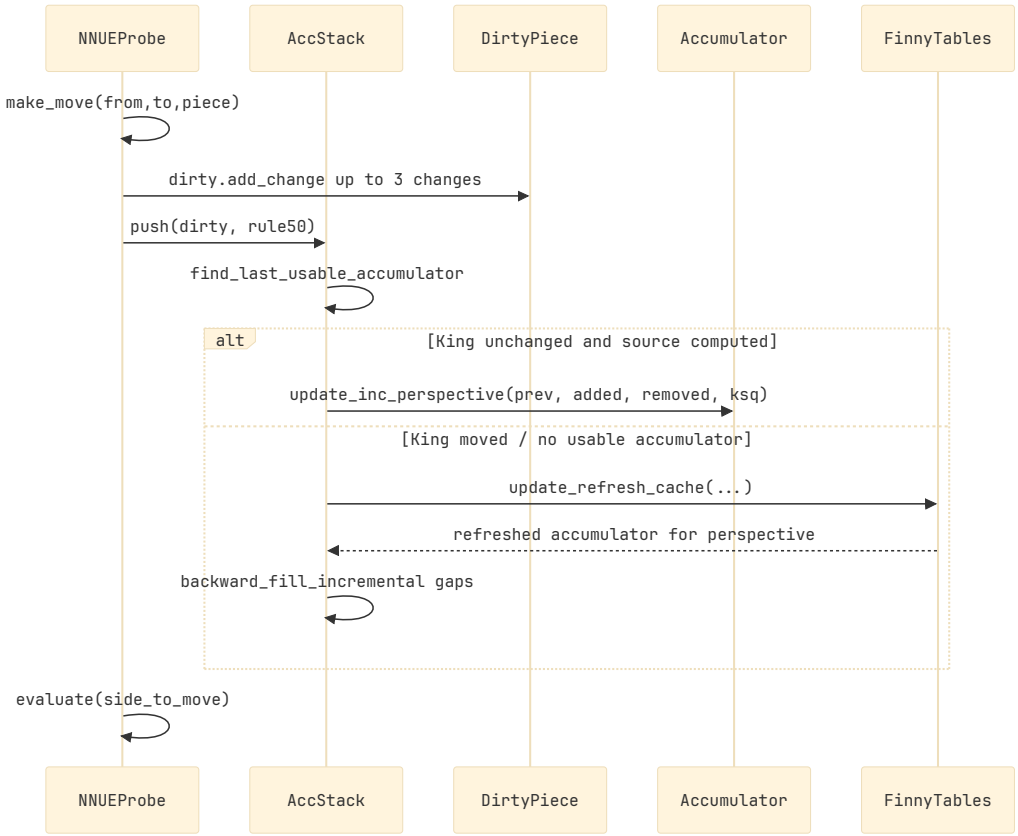

Figure 15. Sequence of operations for incremental evaluation update

The sequence highlights the core performance property of NNUE: update only features affected by a move, instead of recomputing a full network input transform from scratch.

4. 4. Strikes Attack-Generation Crate

`strikes` provides precomputed attack and path lookup infrastructure for efficient bitboard move generation. The crate includes fixed tables for non-sliding pieces and generated lookup tables for sliding pieces.

4.1. 4.1 Attack-Generation Design

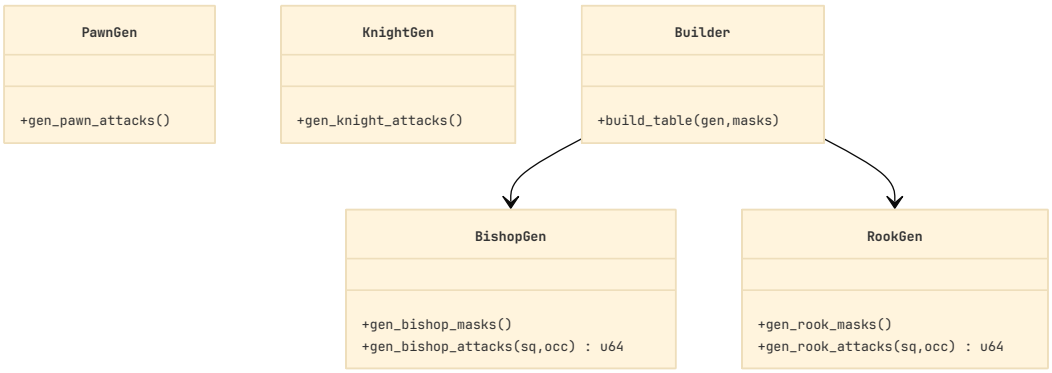

Figure 16. High-level attack-generation architecture in strikes

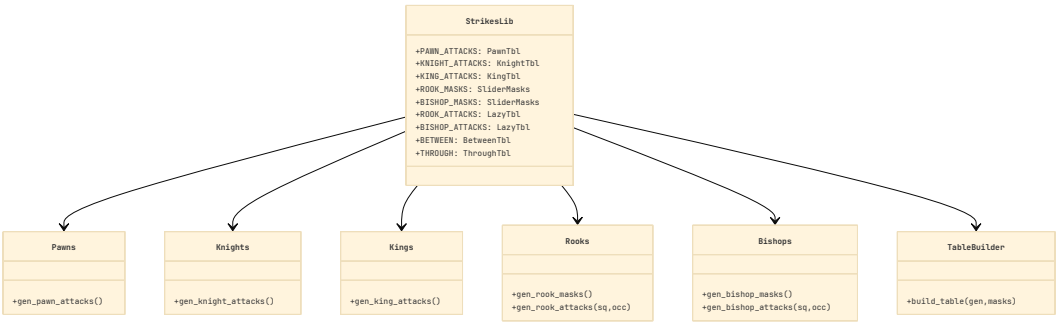

Figure 17. Attack-table and mask ownership for piece move lookups

The implementation exposes static attack resources and lazy-initialized slider tables, supporting fast repeated lookups in search-heavy contexts.

4.2. 4.2 Rays, Paths, and Lookup Pipeline

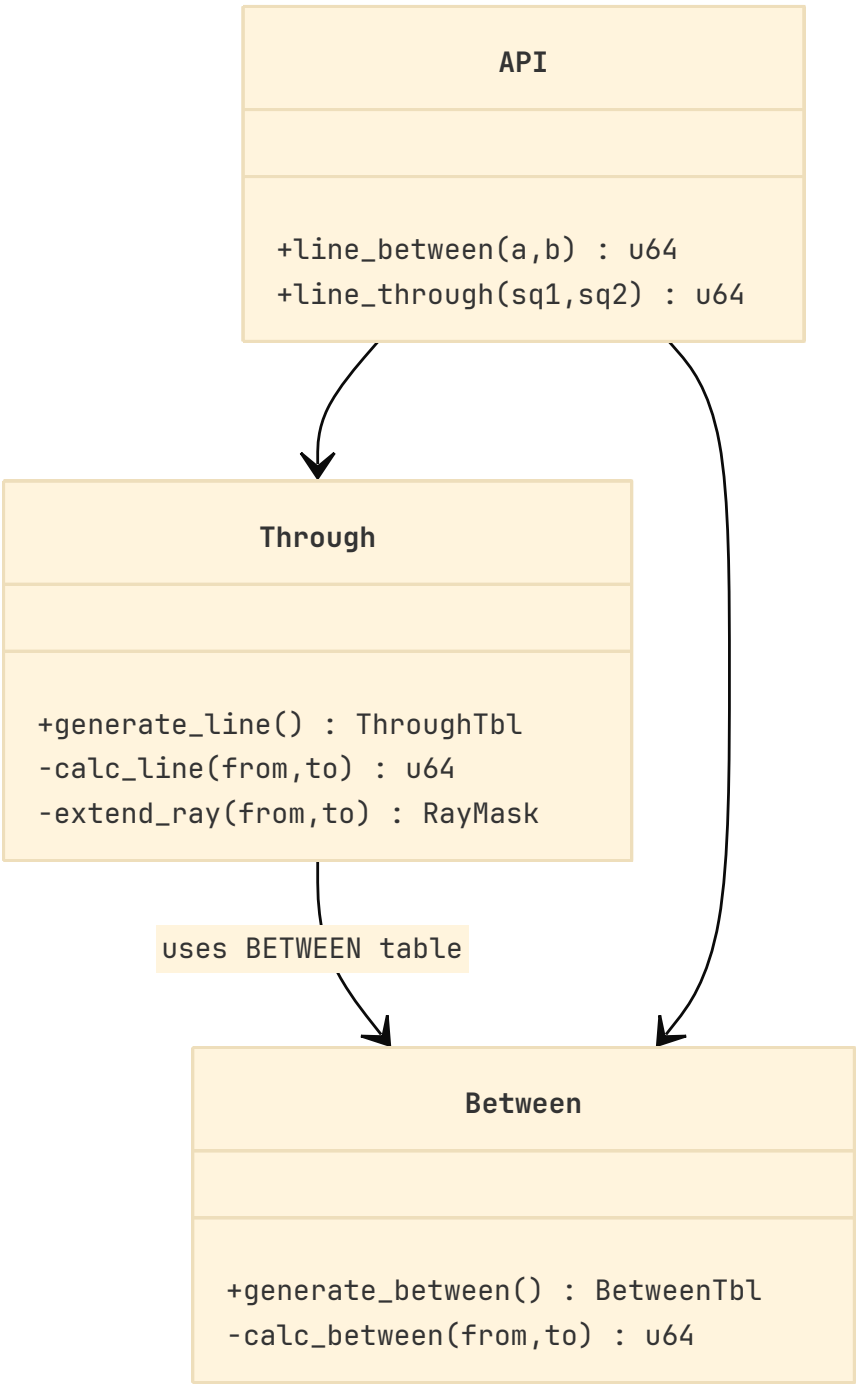

Figure 18. Path and ray abstractions used for between/through queries

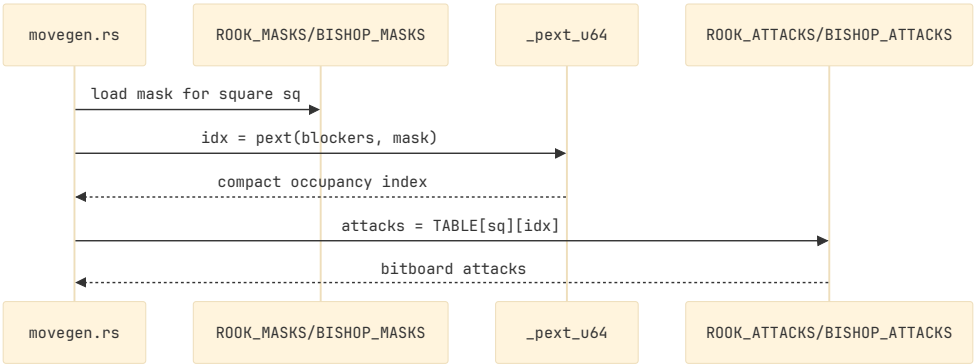

Figure 19. Sliding-piece lookup flow using mask extraction and index mapping

Path queries (`between` / `through`) support legality checks and pin-related logic, while the lookup sequence captures the optimized flow used for sliding attacks.

4.3. 4.3 Initialization Strategy

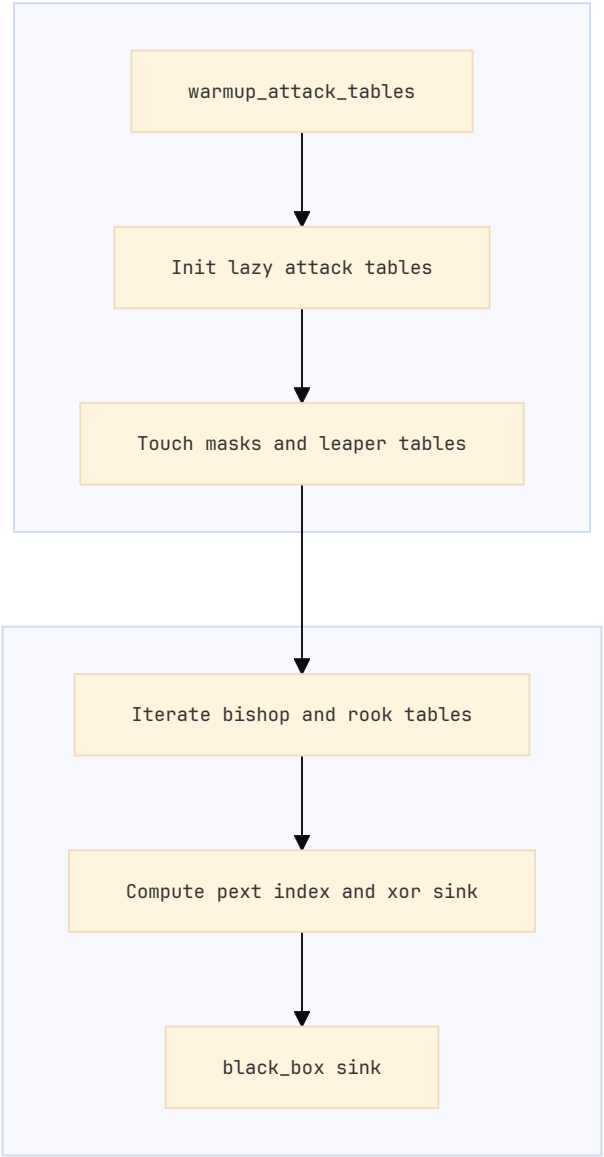

Figure 20. Warmup activity for initializing and touching attack tables

`warmup_attack_tables()` is called during engine startup to force lazy table materialization and reduce first-search latency.

5. 5. Utilities Support Crate

`utilities` contains shared helper functionality for square notation and board visualization. Although small in size, it improves readability and consistency across debugging, test, and development workflows.

5.1. 5.1 Utility Class Structure

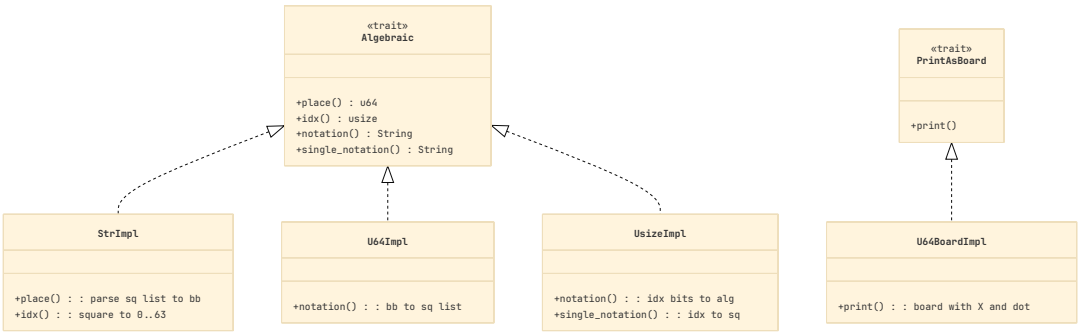

Figure 21. Class-level structure of notation and board utility helpers

5.2. 5.2 Parsing and Rendering Activities

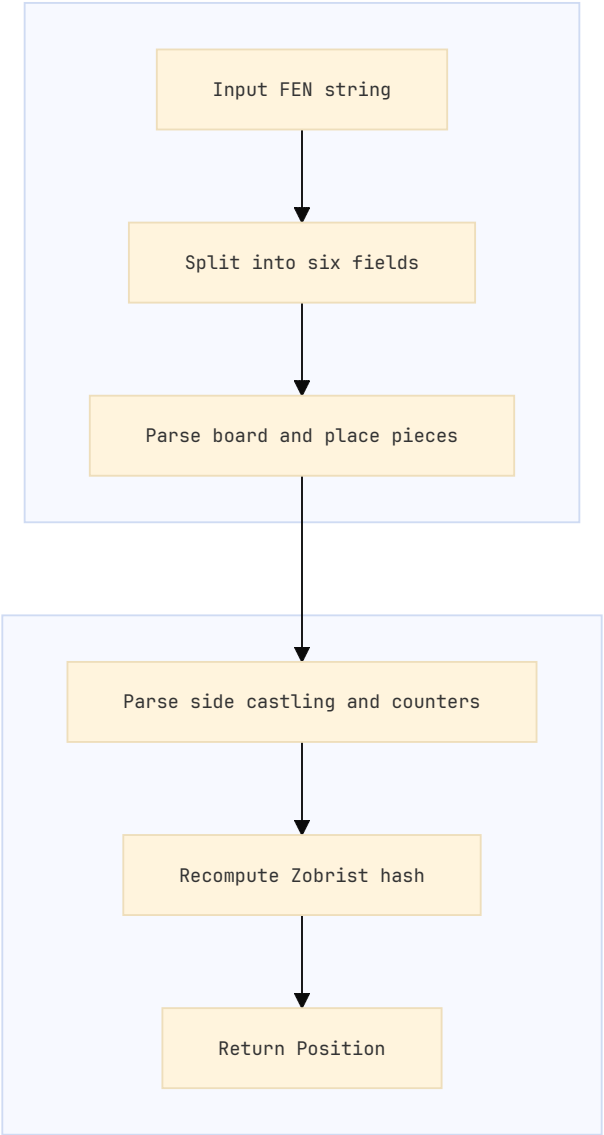

Figure 22. Activity flow for FEN parsing into internal board state

 Artefact Report

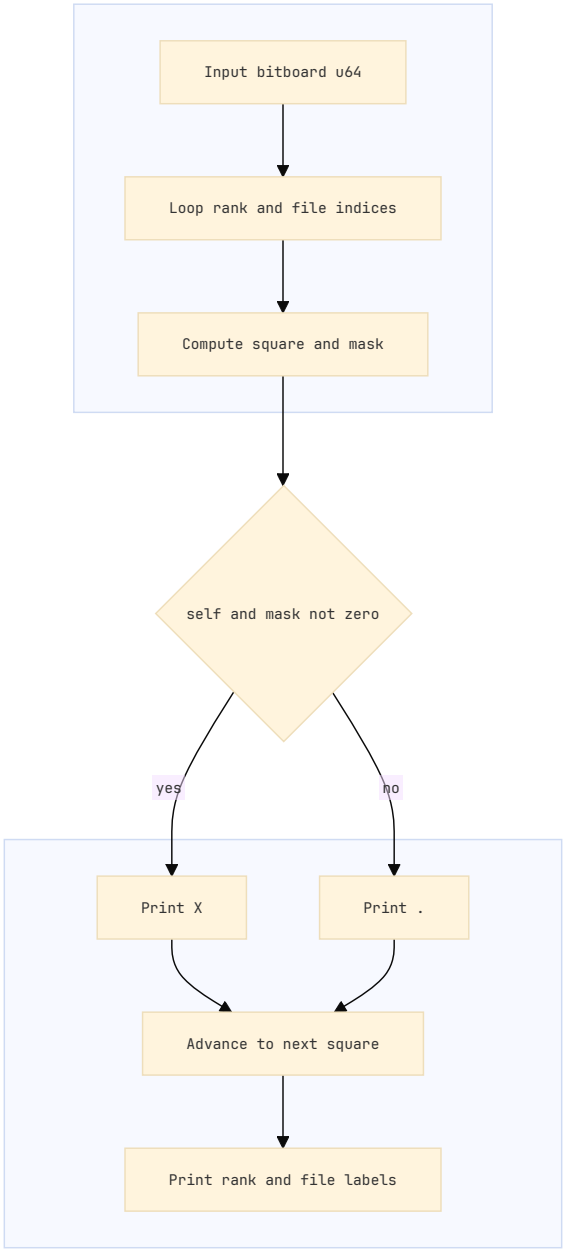

Figure 23. Activity flow for board-state visualization utilities

These utility flows reduce incidental complexity in higher-level modules by keeping parsing and formatting concerns local to dedicated helpers.

6. 6. Cross-Cutting Architecture

This section consolidates subsystem relationships and build-time dependencies across the project.

6.1. 6.1 System Architecture View

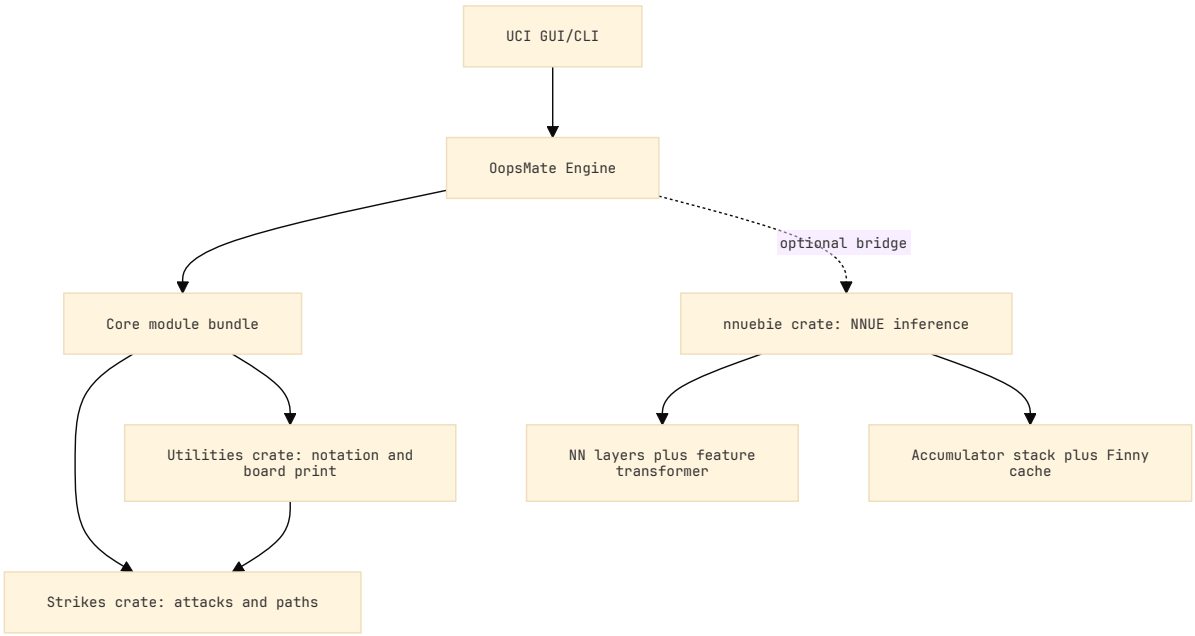

Figure 24. System-wide architecture across OopsMate, nnuebie, strikes, and utilities

6.2. 6.2 Dependency View

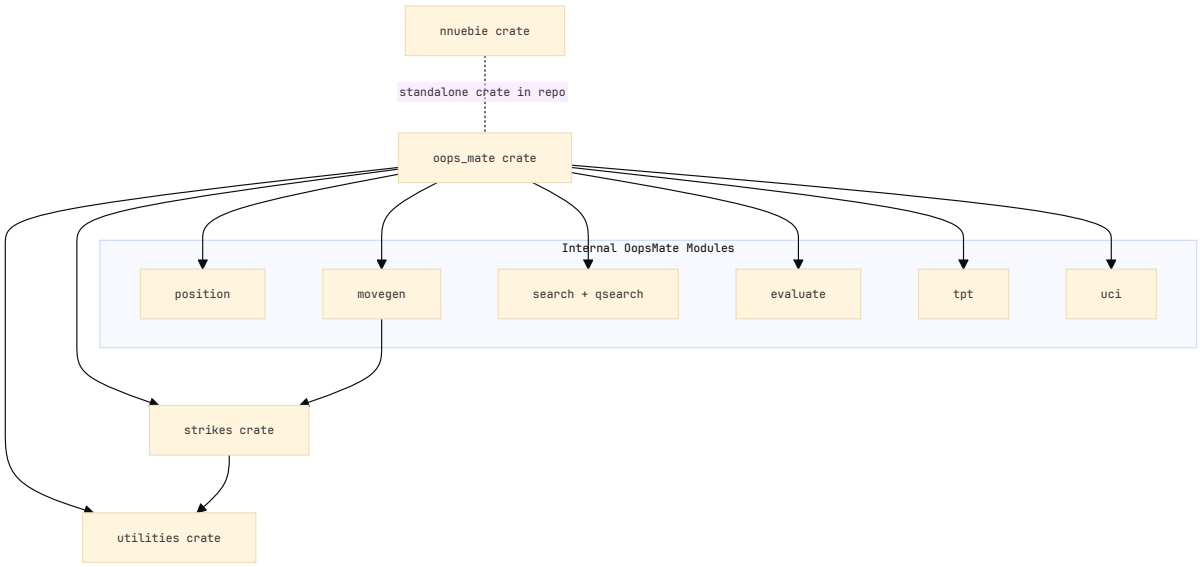

Figure 25. Dependency graph of engine and support crates

From the current codebase:

- `oops_mate` directly depends on `strikes` and `utilities`.
- `strikes` provides attack/path tables consumed by move generation.

- `utilities` provides notation and board helpers used by supporting logic and tests.
- `nnuebie` is maintained as a separate crate intended for engine-side evaluation integration through piece-square feature interfaces.

7. 7. Conclusion

The artefact demonstrates a modular Rust architecture for chess-engine development, with clear boundaries between protocol/search control (`OopsMate`), move-generation infrastructure (`strikes`), support helpers (`utilities`), and neural evaluation capability (`nnuebie`).

The diagram set captures both static design and dynamic behavior, enabling implementation traceability and providing a reliable basis for future extension work such as deeper NNUE integration, search-heuristic experimentation, and benchmarking automation.